Iptables and Basics of the Linux Kernel Firewall

Netfilter, packet traversal, stateful firewalling, NAT and Docker traffic flows

Practical lab: continue the Docker bridge example with nginx, published ports and firewall rules

Agenda

A firewall-focused continuation of the container networking introduction

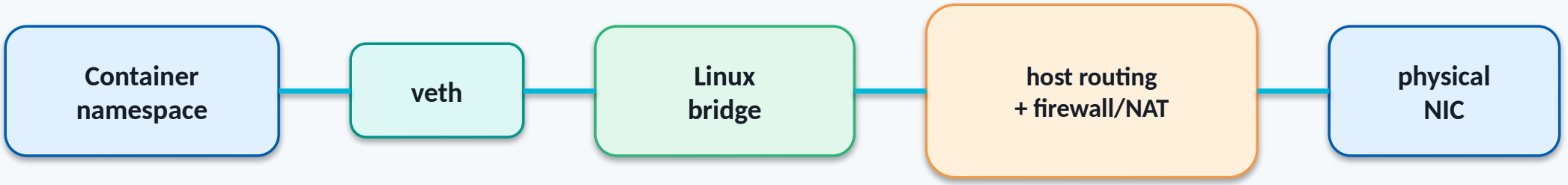
- Kernel firewall architecture: Netfilter, iptables and nftables
- Packet traversal: INPUT, FORWARD, OUTPUT, PREROUTING, POSTROUTING
- Filtering: chains, tables, rule order and default policies
- Stateful firewalling: conntrack states and session tracking
- NAT: SNAT, MASQUERADE, DNAT and Docker port publishing
- Practical lab: secure and troubleshoot a Docker nginx service

Learning outcomes

What you should be able to do after this deck

- Draw the Linux firewall packet path from memory
- Know which chain to inspect for host, forwarded and container traffic
- Read basic iptables rules and iptables-save output
- Build a simple stateful host firewall
- Explain how Docker uses NAT and FORWARD rules
- Troubleshoot a published container port using iptables, conntrack and tcpdump

Recap: container networking mental model



This deck zooms into the host routing + firewall/NAT box.

Why Linux firewalling matters for containers

A container problem is often a Linux firewall path problem

- Containers use Linux networking primitives, not a separate network stack
- Docker creates firewall and NAT rules automatically
- Published ports are implemented through kernel packet traversal
- Outbound container Internet access commonly depends on MASQUERADE
- Troubleshooting requires knowing where packets hit firewall chains

Netfilter: the kernel firewall framework

Do not think of iptables as the firewall engine itself

- Netfilter is built into the Linux kernel packet path
- It provides packet filtering, NAT, mangling and connection tracking
- iptables is a user-space tool that configures Netfilter rules
- nftables is the modern framework that is replacing iptables
- The kernel processes packets; tools only configure the rules

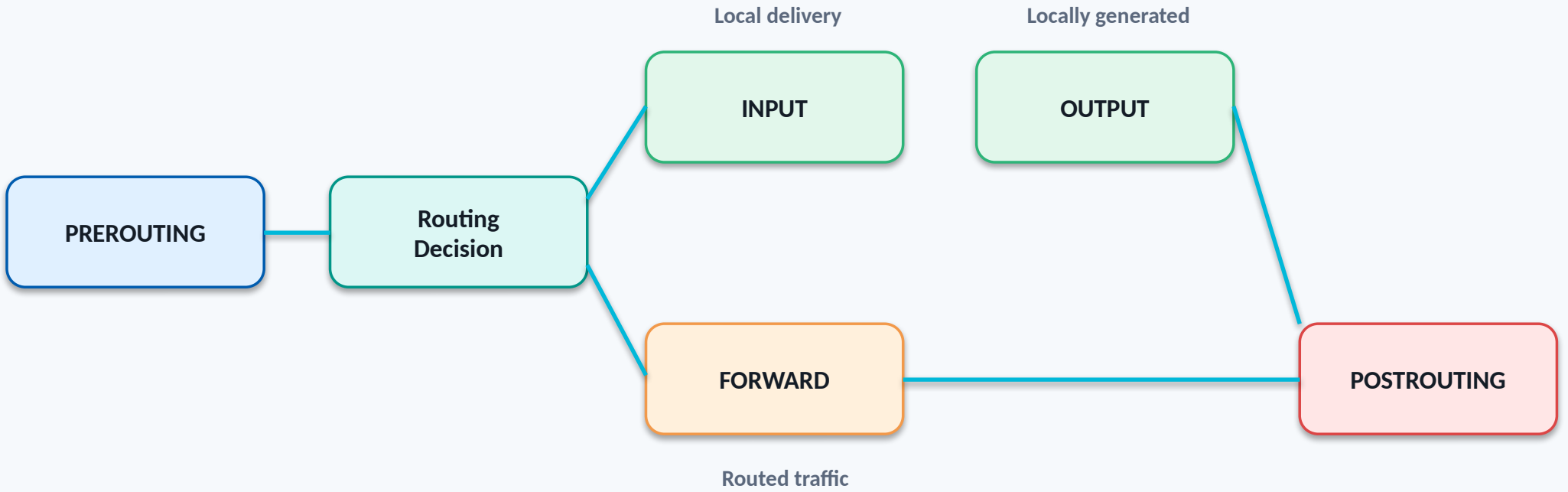
Tooling overview

Tool	Role	Typical use
iptables	classic rule management	filtering and NAT examples
iptables-save	full ruleset export	best visibility tool for iptables
nft	modern ruleset management	nftables-based systems
conntrack	session/NAT state inspection	stateful troubleshooting
tcpdump	packet evidence	prove path before/after firewall

Cisco analogy: ACLs + NAT + stateful inspection

Cisco/networking concept	Linux concept	Important note
ACL entry	iptables rule	Rule order matters
Interface direction	chain/hook position	INPUT/FORWARD/OUTPUT paths
NAT overload/PAT	MASQUERADE/SNAT	Often in POSTROUTING
Static port forward	DNAT	Often in PREROUTING
Stateful firewall	conntrack	Return traffic matched by state

The most important concept: packet traversal



Most mistakes happen because the rule is correct — but placed in the wrong chain.

Common Traffic Types and Their Firewall Paths

Traffic type	Example	Primary chain/path
Traffic to the Linux host	SSH to host	INPUT
Traffic routed through the host	container → Internet	FORWARD + POSTROUTING
Traffic generated by the host	host curl example.com	OUTPUT + POSTROUTING
Traffic to a published container port	client → host:8080 → container:80	PREROUTING DNAT + FORWARD

Docker traffic is usually forwarded traffic, not local host INPUT traffic.

INPUT vs FORWARD

INPUT
packet is destined for the Linux host

FORWARD
packet is routed through the Linux host

Example: SSH to host → INPUT

Example: published Docker port → FORWARD

iptables tables

Table	Purpose	Common chains
filter	allow/drop/reject traffic	INPUT, FORWARD, OUTPUT
nat	source/destination NAT	PREROUTING, OUTPUT, POSTROUTING
mangle	mark/modify packets	multiple hooks
raw	conntrack exemptions	PREROUTING, OUTPUT

Chains: where rules are evaluated

Chains are packet path checkpoints

- PREROUTING: before the routing decision, common place for DNAT
- INPUT: packets delivered locally to the Linux host
- FORWARD: routed packets passing through the Linux host
- OUTPUT: packets generated by local processes
- POSTROUTING: after routing decision, common place for SNAT/MASQUERADE

Basic iptables rule syntax

Terminal

```
# allow SSH to the host
sudo iptables -A INPUT -p tcp --dport 22 -j ACCEPT

# drop ICMP to the host
sudo iptables -A INPUT -p icmp -j DROP

# list rules with counters
sudo iptables -L -n -v
```

Rule = chain + match conditions + target/action

Common match conditions

Match	Example	Meaning
Protocol	-p tcp	match TCP packets
Source	-s 10.0.0.0/24	match source network
Destination	-d 192.168.1.10	match destination IP
Port	--dport 443	match destination TCP/UDP port
Interface	-i eth0 / -o docker0	match ingress or egress interface
State	-m conntrack --ctstate ESTABLISHED	match connection state

Common targets/actions

Target	Meaning	Typical use
ACCEPT	allow the packet	permit required traffic
DROP	silently discard	stealthier blocking
REJECT	discard and notify sender	clear troubleshooting behavior
LOG	log and continue	visibility before decision
DNAT	change destination	published ports
MASQUERADE	dynamic source NAT	container egress

Rule evaluation order

This is similar to ACL processing, but across multiple chains

- Rules are evaluated top-down inside a chain
- The first terminating match usually decides the packet outcome
- Specific exceptions should appear before broad rules
- Default policy applies if no rule matches
- Rule counters help prove which rule is being hit

Use iptables-save for visibility

Terminal

```
$ sudo iptables-save
*filter
:INPUT ACCEPT [0:0]
:FORWARD DROP [0:0]
:OUTPUT ACCEPT [0:0]
-A INPUT -p tcp -m tcp --dport 22 -j ACCEPT
COMMIT
*nat
:PREROUTING ACCEPT [0:0]
:POSTROUTING ACCEPT [0:0]
COMMIT
```

Default policies

Terminal

```
# default deny for host inbound traffic
sudo iptables -P INPUT DROP
sudo iptables -P FORWARD DROP
sudo iptables -P OUTPUT ACCEPT

# but first allow existing sessions and management access
```

Warning: setting INPUT DROP remotely without allowing SSH can lock you out.

Stateful firewalling with conntrack

Stateful inspection is the firewall memory

- conntrack tracks flows and connection state
- Stateful rules allow return traffic without reverse static rules
- Common states: NEW, ESTABLISHED, RELATED, INVALID
- NAT depends on conntrack to map return traffic
- conntrack is central to Docker NAT behavior

conntrack states

State	Meaning	Typical firewall action
NEW	new connection attempt	allow only if explicitly permitted
ESTABLISHED	part of an existing flow	allow
RELATED	related to an existing flow	allow when needed
INVALID	cannot be identified or malformed state	drop/log

Basic stateful host firewall pattern

Terminal

```
# allow loopback
sudo iptables -A INPUT -i lo -j ACCEPT

# allow return traffic
sudo iptables -A INPUT -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT

# allow SSH
sudo iptables -A INPUT -p tcp --dport 22 -j ACCEPT

# default deny
sudo iptables -P INPUT DROP
```

Inspecting conntrack

Terminal

```
$ sudo conntrack -L
tcp 6 431999 ESTABLISHED src=10.0.0.50 dst=10.0.0.20 sport=51522 dport=22 ...

$ sudo conntrack -L -p tcp --dport 80
$ sudo conntrack -E

# conntrack -E streams events live
```

When NAT is involved, conntrack shows the before/after tuple.

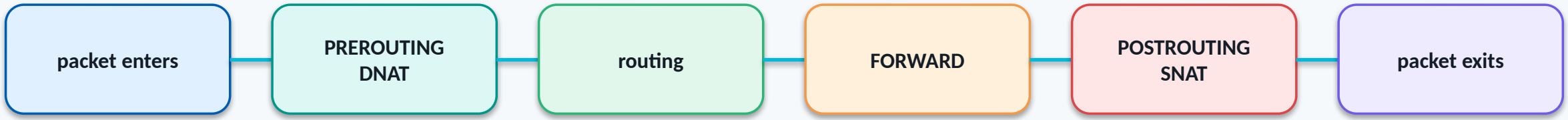
NAT in Linux

NAT is packet rewriting plus state tracking

- NAT rules live in the nat table
- DNAT commonly happens before routing in PREROUTING
- SNAT/MASQUERADE commonly happens after routing in POSTROUTING
- NAT is applied mainly to the first packet of a connection
- conntrack applies the remembered translation to the rest of the flow

NAT flow overview

Where translations normally occur



DNAT changes where the packet goes. SNAT changes who the packet appears to come from.

SNAT and MASQUERADE

Terminal

```
# static source NAT
sudo iptables -t nat -A POSTROUTING -s 172.18.0.0/16 -o eth0 \
  -j SNAT --to-source 10.0.0.20

# dynamic source NAT
sudo iptables -t nat -A POSTROUTING -s 172.18.0.0/16 -o eth0 \
  -j MASQUERADE
```

Docker commonly uses MASQUERADE for container egress.

DNAT for port forwarding

Terminal

```
# forward host TCP/8080 to container TCP/80
sudo iptables -t nat -A PREROUTING -p tcp --dport 8080 \
  -j DNAT --to-destination 172.18.0.2:80

# allow forwarded traffic
sudo iptables -A FORWARD -p tcp -d 172.18.0.2 --dport 80 -j ACCEPT
```

Port publishing is a DNAT + forwarding problem.

Docker and iptables

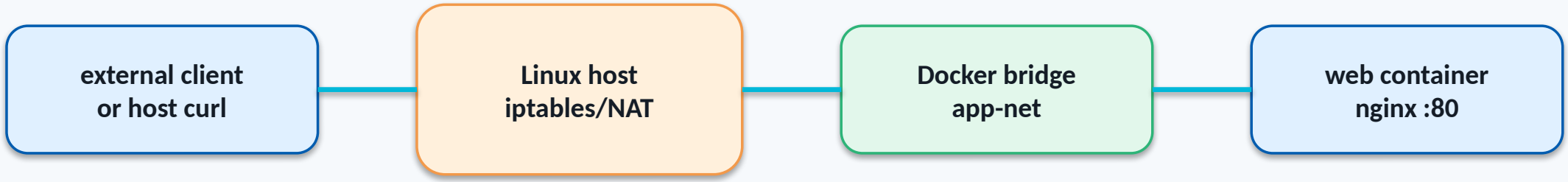
Docker is not bypassing Linux firewalling — it is using it

- Docker creates custom chains for its bridge networks and published ports
- Docker modifies nat and filter tables automatically
- DOCKER chain commonly contains DNAT rules for published ports
- DOCKER-USER is intended for administrator-defined policy
- Do not edit Docker-managed chains blindly

Docker chain idea

Exact chains may vary by Docker version, iptables backend and distribution.

Practical lab topology



We continue the Docker bridge example: app-net + nginx + published port 8080.

Lab step 1: create app network and web container

Terminal

```
$ docker network create app-net
$ docker run -d --name web --network app-net -p 8080:80 nginx:alpine

$ docker ps --format 'table {{.Names}}\t{{.Ports}}\t{{.Networks}}'
NAMES   PORTS                   NETWORKS
web     0.0.0.0:8080->80/tcp    app-net
```

Lab step 2: verify application and container IP

Terminal

```
$ curl -I http://127.0.0.1:8080
HTTP/1.1 200 OK

$ docker inspect -f '{{range.NetworkSettings.Networks}}{{.IPAddress}}{{end}}' web
172.18.0.2

$ docker exec web ss -ltnp
LISTEN 0.0.0.0:80
```

Lab step 3: inspect Docker-generated NAT

Terminal

```
$ sudo iptables-save | grep -E '8080|DNAT|MASQUERADE|DOCKER'
-A PREROUTING -m addrtype --dst-type LOCAL -j DOCKER
-A DOCKER ! -i br-xxxx -p tcp -m tcp --dport 8080 \
    -j DNAT --to-destination 172.18.0.2:80
-A POSTROUTING -s 172.18.0.0/16 ! -o br-xxxx -j MASQUERADE
```

This single output shows both inbound DNAT and outbound MASQUERADE.

Lab step 4: prove the packet path with tcpdump

Terminal

```
# terminal 1: capture host port
$ sudo tcpdump -ni any tcp port 8080

# terminal 2: capture container port on Docker bridge
$ sudo tcpdump -ni br-xxxx tcp port 80

# terminal 3: generate traffic
$ curl http://127.0.0.1:8080
```

Before DNAT: host:8080. After DNAT: container:80.

Lab step 5: inspect conntrack

Terminal

```
$ sudo conntrack -L -p tcp | grep 172.18.0.2
tcp 6 431999 ESTABLISHED src=127.0.0.1 dst=127.0.0.1 sport=51522 dport=8080
    src=172.18.0.2 dst=172.18.0.1 sport=80 dport=51522

# exact output depends on source path and Docker settings
```

conntrack shows the remembered flow state used for return traffic.

Lab step 6: add a simple DOCKER-USER policy

Terminal

```
# example: allow one source, then drop other forwarded container ingress
sudo iptables -I DOCKER-USER 1 -s 10.0.0.50 -p tcp --dport 80 -j ACCEPT
sudo iptables -A DOCKER-USER -p tcp --dport 80 -j DROP

# inspect counters
sudo iptables -L DOCKER-USER -n -v --line-numbers
```

Use DOCKER-USER for custom Docker filtering instead of editing Docker-managed chains.

Lab safety: remove test rules

Terminal

```
$ sudo iptables -L DOCKER-USER -n -v --line-numbers

# delete by line number, starting with the highest number
$ sudo iptables -D DOCKER-USER 2
$ sudo iptables -D DOCKER-USER 1

# or flush only if you know this is a lab host
$ sudo iptables -F DOCKER-USER
```

Never casually flush production firewall rules.

Troubleshooting workflow: published port

1. **docker ps**
   is port published?
2. **ss in container**
   is app listening?
3. **iptables nat**
   is DNAT present?
4. **iptables filter**
   is FORWARD allowed?
5. **conntrack**
   is state created?
6. **tcpdump**
   where does packet stop?

Scenario: rule in the wrong chain

Terminal

```
# admin tries to block published container port
sudo iptables -A INPUT -p tcp --dport 8080 -j DROP

# but Docker published port may still work
$ curl http://host-ip:8080
HTTP/1.1 200 OK

! published container traffic is commonly FORWARD after DNAT
```

Correct-looking rule, wrong path.

Scenario: NAT is fine, application is broken

Terminal

```
$ docker ps
web  0.0.0.0:8080->80/tcp

$ curl http://127.0.0.1:8080
curl: Empty reply from server

$ docker exec web ss -ltnp
# no listener on :80
```

Do not blame iptables until you prove the service is listening.

Scenario: container has no Internet access

Terminal

```
$ docker exec client ping -c 2 8.8.8.8
100% packet loss

# check route in container
$ docker exec client ip route

# check host forwarding and NAT
$ sysctl net.ipv4.ip_forward
$ iptables-save | grep MASQUERADE
```

Likely areas: container default route, host forwarding, FORWARD chain or POSTROUTING MASQUERADE.

Logging packets safely

Terminal

```
# log before dropping
sudo iptables -A INPUT -p tcp --dport 8080 -j LOG --log-prefix 'FW DROP 8080: '
sudo iptables -A INPUT -p tcp --dport 8080 -j DROP

# view logs
sudo journalctl -k -f
sudo dmesg -w
```

Be careful: logging high-volume traffic can overwhelm logs.

nftables reality check

Learn the packet model first; syntax can change

- Many modern distributions use nftables under the hood
- iptables commands may use the iptables-nft compatibility layer
- iptables-save may still show rules even when nftables backend is active
- nft list ruleset shows the native nftables view
- Concepts remain the same: hooks, chains, rules, state, NAT

Inspect nftables

Terminal

```
$ sudo nft list ruleset
table ip nat {
  chain PREROUTING { type nat hook prerouting priority dstnat; }
  chain POSTROUTING { type nat hook postrouting priority srcnat; }
}

$ iptables --version
iptables v1.8.x (nf_tables)
```

Security best practices

Firewalling is both control and visibility

- Start with a clear policy: what should be reachable, from where?
- Use stateful rules for return traffic
- Expose only required Docker ports
- Bind published ports to 127.0.0.1 when external access is not needed
- Use DOCKER-USER for administrator Docker firewall policy
- Always test with tcpdump and rule counters

Lab cleanup

Terminal

```
$ docker rm -f web client
$ docker network rm app-net

# remove only lab rules from DOCKER-USER
$ sudo iptables -L DOCKER-USER -n -v --line-numbers
$ sudo iptables -D DOCKER-USER <line-number>

# verify
$ docker ps -a
$ docker network ls
```

Command cheat sheet: iptables

```
                                                Cheat sheet
iptables -L -n -v
iptables -S
iptables-save
iptables -t nat -L -n -v
iptables -A INPUT -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT
iptables -A INPUT -p tcp --dport 22 -j ACCEPT
iptables -t nat -A POSTROUTING -s 172.18.0.0/16 -j MASQUERADE
iptables -L DOCKER-USER -n -v --line-numbers
```

Command cheat sheet: troubleshooting

Cheat sheet

```
docker ps
docker network inspect app-net
docker exec web ss -ltnp
conntrack -L
conntrack -E
tcpdump -ni any tcp port 8080
tcpdump -ni br-xxxx tcp port 80
ip route get <destination>
bridge fdb show
journalctl -k -f
```

Final mental model

Container traffic is usually not special.
It is Linux forwarded traffic with extra automation.

Outbound	FORWARD → POSTROUTING MASQUERADE
Inbound published port	PREROUTING DNAT → FORWARD → container
Host service	INPUT

Questions?

Packet traversal + conntrack + NAT = Linux firewall troubleshooting

Follow the packet. Check the correct chain. Prove it with counters and captures.